

JavaScript 和 HTML DOM 参考手册

方法介绍、用法罗列与对比记忆

基于 runoob.com 教程

2026 年 1 月 27 日

目录

1 JavaScript 对象参考手册

1.1 Date 对象

1.1.1 Date 对象方法总览

方法	描述	用法示例
getDate()	从 Date 对象返回一个月中的某一天 (1 - 31)	date.getDate()
getDay()	从 Date 对象返回一周中的某一天 (0 - 6)	date.getDay()
getFullYear()	从 Date 对象以四位数字返回年份	date.getFullYear()
getHours()	返回 Date 对象的小时 (0 - 23)	date.getHours()
getMilliseconds()	返回 Date 对象的毫秒 (0 - 999)	date.getMilliseconds()
getMinutes()	返回 Date 对象的分钟 (0 - 59)	date.getMinutes()
getMonth()	从 Date 对象返回月份 (0 - 11)	date.getMonth()
getSeconds()	返回 Date 对象的秒数 (0 - 59)	date.getSeconds()
getTime()	返回 1970 年 1 月 1 日至今的毫秒数	date.getTime()
getTimezoneOffset()	返回本地时间与格林威治标准时间 (GMT) 的分钟差	date.getTimezoneOffset()
getUTCDate()	根据世界时从 Date 对象返回月中的一天 (1 - 31)	date.getUTCDate()
getUTCDay()	根据世界时从 Date 对象返回周中的一天 (0 - 6)	date.getUTCDay()
getUTCFullYear()	根据世界时从 Date 对象返回四位数的年份	date.getUTCFullYear()
getUTCHours()	根据世界时返回 Date 对象的小时 (0 - 23)	date.getUTCHours()
getUTCMilliseconds()	根据世界时返回 Date 对象的毫秒 (0 - 999)	date.getUTCMilliseconds()

<code>getUTCMinutes()</code>	根据世界时返回 Date 对象的分钟 (0 - 59)	<code>date.getUTCMinutes()</code>
<code>getUTCMonth()</code>	根据世界时从 Date 对象返回月份 (0 - 11)	<code>date.getUTCMonth()</code>
<code>getUTCSeconds()</code>	根据世界时返回 Date 对象的秒钟 (0 - 59)	<code>date.getUTCSeconds()</code>
<code>parse()</code>	返回 1970 年 1 月 1 日午夜到指定日期（字符串）的毫秒数	<code>Date.parse(dateString)</code>
<code>setDate()</code>	设置 Date 对象中月的某一天 (1 - 31)	<code>date.setDate(day)</code>
<code>setFullYear()</code>	设置 Date 对象中的年份（四位数字）	<code>date.setFullYear(year, month, day)</code>
<code>setHours()</code>	设置 Date 对象中的小时 (0 - 23)	<code>date.setHours(hour, min, sec, ms)</code>
<code>setMilliseconds()</code>	设置 Date 对象中的毫秒 (0 - 999)	<code>date.setMilliseconds(ms)</code>
<code>setMinutes()</code>	设置 Date 对象中的分钟 (0 - 59)	<code>date.setMinutes(min, sec, ms)</code>
<code>setMonth()</code>	设置 Date 对象中月份 (0 - 11)	<code>date.setMonth(month, day)</code>
<code>setSeconds()</code>	设置 Date 对象中的秒钟 (0 - 59)	<code>date.setSeconds(sec, ms)</code>
<code>setTime()</code>	以毫秒设置 Date 对象	<code>date.setTime(milliseconds)</code>
<code>setUTCDate()</code>	根据世界时设置 Date 对象中月份的一天 (1 - 31)	<code>date.setUTCDate(day)</code>
<code>setUTCFullYear()</code>	根据世界时设置 Date 对象中的年份（四位数字）	<code>date.setUTCFullYear(year, month, day)</code>
<code>setUTCHours()</code>	根据世界时设置 Date 对象中的小时 (0 - 23)	<code>date.setUTCHours(hour, min, sec, ms)</code>
<code>setUTCMilliseconds()</code>	根据世界时设置 Date 对象中的毫秒 (0 - 999)	<code>date.setUTCMilliseconds(ms)</code>
<code>setUTCMinutes()</code>	根据世界时设置 Date 对象中的分钟 (0 - 59)	<code>date.setUTCMinutes(min, sec, ms)</code>

setUTCMonth()	根据世界时设置 Date 对象中的月份 (0 - 11)	date.setUTCMonth(month, day)
setUTCSeconds()	根据世界时 (UTC) 设置指定时间的秒字段	date.setUTCSeconds(sec, ms)
toDateString()	把 Date 对象的日期部分转换为字符串	date.toDateString()
toISOString()	使用 ISO 标准返回字符串的日期格式	date.toISOString()
toJSON()	以 JSON 数据格式返回日期字符串	date.toJSON()
toLocaleDateString()	根据本地时间格式, 把 Date 对象的日期部分转换为字符串	date.toLocaleDateString()
toLocaleTimeString()	根据本地时间格式, 把 Date 对象的时间部分转换为字符串	date.toLocaleTimeString()
toLocaleString()	根据本地时间格式, 把 Date 对象转换为字符串	date.toLocaleString()
toString()	把 Date 对象转换为字符串	date.toString()
toTimeString()	把 Date 对象的时间部分转换为字符串	date.toTimeString()
toUTCString()	根据世界时, 把 Date 对象转换为字符串	date.toUTCString()
UTC()	根据世界时返回 1970 年 1 月 1 日到指定日期的毫秒数	Date.UTC(year, month, day, hour, ...)
valueOf()	返回 Date 对象的原始值	date.valueOf()
now()	返回当前时间的毫秒数	Date.now()

1.1.2 Date 对象方法对比与记忆技巧

获取方法对比

- 本地时间获取方法: getDate(), getDay(), getFullYear(), getHours(), getMilliseconds(), getMinutes(), getMonth(), getSeconds()
- UTC 时间获取方法: getUTCDate(), getUTCDay(), getUTCFullYear(), getUTCHours(), getUTCMilliseconds(), getUTCMinutes(), getUTCMonth(), getUTCSeconds()

设置方法对比

- **本地时间设置方法:** setDate(), setFullYear(), setHours(), setMilliseconds(), setMinutes(), setMonth(), setSeconds(), setTime()
- **UTC 时间设置方法:** setUTCDate(), setUTCFullYear(), setUTCHours(), setUTCMilliseconds(), setUTCMinutes(), setUTCMonth(), setUTCSeconds()

转换方法对比

- **日期部分转换:** toString(), toLocaleDateString()
- **时间部分转换:** toTimeString(), toLocaleTimeString()
- **完整转换:** toString(), toLocaleString(), toUTCString()
- **特殊格式转换:** toISOString(), toJSON()

记忆技巧

1. **命名规律:** 所有获取方法以 get 开头, 设置方法以 set 开头, UTC 相关方法在中间添加 UTC
2. **参数范围:** 月份是 0-11 (需要加 1 才是实际月份), 星期是 0-6 (0 表示星期日)
3. **时间戳:** getTime() 和 now() 返回的是时间戳 (毫秒数), 可用于时间计算
4. **静态方法:** parse(), UTC(), now() 是静态方法, 直接通过 Date 调用
5. **转换方法:** 根据需求选择合适的转换方法, 本地时间用 toLocale*, UTC 时间用 toUTC*

1.1.3 Date 对象实例用法

```
1 // 创建 Date 对象的四种方法
2 var d1 = new Date();
3 var d2 = new Date(milliseconds); // 参数为毫秒
4 var d3 = new Date(dateString);
5 var d4 = new Date(year, month, day, hours, minutes, seconds, milliseconds);
6
7 // 示例: 获取当前日期和时间
8 var today = new Date();
9 var year = today.getFullYear();
10 var month = today.getMonth() + 1; // 月份需要加 1
11 var day = today.getDate();
12 var hours = today.getHours();
13 var minutes = today.getMinutes();
14 var seconds = today.getSeconds();
15
16 // 示例: 设置日期和时间
17 var future = new Date();
18 future.setFullYear(2025, 11, 31); // 2025年12月31日
19 future.setHours(23, 59, 59); // 23:59:59
20
21 // 示例: 日期转换为字符串
```

```
22 console.log(today.toLocaleString()); // 本地时间字符串
23 console.log(today.toUTCString()); // UTC时间字符串
24 console.log(today.toISOString()); // ISO格式字符串
25
26 // 示例：计算时间差
27 var start = new Date();
28 // 执行一些操作
29 var end = new Date();
30 var diff = end.getTime() - start.getTime(); // 时间差（毫秒）
31 console.log("操作耗时: " + diff + "毫秒");
```

Listing 1: Date 对象使用示例

1.2 Array 对象

1.2.1 Array 对象方法总览

方法	描述	用法示例
concat()	连接两个或更多的数组，并返回结果	<code>array1.concat(array2, array3, ...)</code>
copyWithin()	从数组的指定位置拷贝元素到数组的另一个指定位置中	<code>array.copyWithin(target, start, ...)</code>
entries()	返回数组的可迭代对象	<code>array.entries()</code>
every()	检测数组元素的每个元素是否都符合条件	<code>array.every(function(currentValue, ...)</code>
fill()	使用一个固定值来填充数组	<code>array.fill(value, start, end)</code>
filter()	检测数组元素，并返回符合条件所有元素的数组	<code>array.filter(function(currentValue, ...)</code>
find()	返回符合传入测试（函数）条件的数组元素	<code>array.find(function(currentValue, ...)</code>
findIndex()	返回符合传入测试（函数）条件的数组元素索引	<code>array.findIndex(function(currentValue, ...)</code>
forEach()	数组每个元素都执行一次回调函数	<code>array.forEach(function(currentValue, ...)</code>
from()	通过给定的对象中创建一个数组	<code>Array.from(object, mapFunction, ...)</code>
includes()	判断一个数组是否包含一个指定的值	<code>array.includes(searchElement, start, ...)</code>
indexOf()	搜索数组中的元素，并返回它所在的位置	<code>array.indexOf(item, start)</code>
isArray()	判断对象是否为数组	<code>Array.isArray(obj)</code>
join()	把数组的所有元素放入一个字符串	<code>array.join(separator)</code>

keys()	返回数组的可迭代对象，包含原始数组的键 (key)	array.keys()
lastIndexOf()	搜索数组中的元素，并返回它最后出现的位置	array.lastIndexOf(item, start)
map()	通过指定函数处理数组的每个元素，并返回处理后的数组	array.map(function(currentValue,
pop()	删除数组的最后一个元素并返回删除的元素	array.pop()
push()	向数组的末尾添加一个或更多元素，并返回新的长度	array.push(item1, item2, ..., it
reduce()	将数组元素计算为一个值（从左到右）	array.reduce(function(total, cur
reduceRight()	将数组元素计算为一个值（从右到左）	array.reduceRight(function(total
reverse()	反转数组的元素顺序	array.reverse()
shift()	删除并返回数组的第一个元素	array.shift()
slice()	从某个已有的数组返回选定的元素	array.slice(start, end)
some()	检测数组元素中是否有元素符合指定条件	array.some(function(currentValue
sort()	对数组的元素进行排序	array.sort(sortfunction)
splice()	从数组中添加或删除元素	array.splice(index, howmany, ite
toString()	把数组转换为字符串，并返回结果	array.toString()
unshift()	向数组的开头添加一个或更多元素，并返回新的长度	array.unshift(item1, item2, ...,
valueOf()	返回数组对象的原始值	array.valueOf()

1.2.2 Array 对象方法对比与记忆技巧

增删改查方法对比

- **增加元素:** `push()` (末尾)、`unshift()` (开头)、`splice()` (指定位置)
- **删除元素:** `pop()` (末尾)、`shift()` (开头)、`splice()` (指定位置)、`slice()` (返回新数组)
- **修改元素:** `splice()`、`copyWithin()`、`fill()`
- **查找元素:** `indexOf()`、`lastIndexOf()`、`includes()`、`find()`、`findIndex()`

迭代方法对比

- **遍历方法:** `forEach()` (无返回值)、`map()` (返回新数组)
- **检测方法:** `every()` (所有元素满足)、`some()` (至少一个元素满足)
- **过滤方法:** `filter()` (返回符合条件的元素)
- **归约方法:** `reduce()` (从左到右)、`reduceRight()` (从右到左)

转换方法对比

- **数组转字符串:** `toString()`、`join()` (可指定分隔符)
- **其他对象转数组:** `Array.from()`

记忆技巧

1. **命名规律:** 以 `ing` 结尾的方法通常会修改原数组 (如 `sorting`、`splicing`)
2. **返回值:** 返回新数组的方法 (如 `map`、`filter`、`slice`、`concat`) 不会修改原数组
3. **参数顺序:** `splice()` 的参数是 `(index, howmany, item1, ...)`, 而 `slice()` 的参数是 `(start, end)`
4. **静态方法:** `isArray()`、`from()` 是静态方法, 直接通过 `Array` 调用
5. **遍历方法:** 需要返回新数组用 `map()`, 不需要返回值用 `forEach()`

1.3 String 对象

1.3.1 String 对象方法总览

方法	描述	用法示例
charAt()	返回指定索引位置的字符	<code>string.charAt(index)</code>
charCodeAt()	返回指定索引位置字符的 Unicode 值	<code>string.charCodeAt(index)</code>
concat()	连接两个或多个字符串，返回连接后的字符串	<code>string.concat(string1, string2, ...)</code>
fromCharCode()	将 Unicode 转换为字符串	<code>String.fromCharCode(n1, n2, ..., n)</code>
indexOf()	返回字符串中检索指定字符第一次出现的位置	<code>string.indexOf(searchvalue, startindex)</code>
lastIndexOf()	从后向前搜索字符串，返回指定字符最后出现的位置	<code>string.lastIndexOf(searchvalue, startindex)</code>
localeCompare()	用本地特定的顺序来比较两个字符串	<code>string.localeCompare(compareString)</code>
match()	查找字符串中匹配正则表达式的内容	<code>string.match(regexp)</code>
replace()	在字符串中查找匹配的子串，并替换与正则表达式匹配的子串	<code>string.replace(searchvalue, newvalue)</code>
search()	查找与正则表达式相匹配的值	<code>string.search(searchvalue)</code>
slice()	提取字符串的片断，并在新的字符串中返回被提取的部分	<code>string.slice(start, end)</code>
split()	把字符串分割为字符串数组	<code>string.split(separator, limit)</code>
substr()	从起始索引号提取字符串中指定数目的字符	<code>string.substr(start, length)</code>
substring()	提取字符串中两个指定的索引号之间的字符	<code>string.substring(from, to)</code>

toLocaleLowerCase()	根据本地主机的语言环境把字符串转换为小写	string.toLocaleLowerCase()
toLocaleUpperCase()	根据本地主机的语言环境把字符串转换为大写	string.toLocaleUpperCase()
toLowerCase()	把字符串转换为小写	string.toLowerCase()
toString()	返回字符串对象值	string.toString()
toUpperCase()	把字符串转换为大写	string.toUpperCase()
trim()	去除字符串两边的空白	string.trim()
valueOf()	返回某个字符串对象的原始值	string.valueOf()
includes()	查找字符串中是否包含指定的子字符串	string.includes(searchvalue, start)
startsWith()	查看字符串是否以指定的子字符串开头	string.startsWith(searchvalue, start)
endsWith()	查看字符串是否以指定的子字符串结尾	string.endsWith(searchvalue, length)
repeat()	复制字符串指定次数，并返回连接后的字符串	string.repeat(count)
padStart()	用另一个字符串填充当前字符串，从字符串的开头开始填充	string.padStart(length, string)
padEnd()	用另一个字符串填充当前字符串，从字符串的末尾开始填充	string.padEnd(length, string)

1.3.2 String 对象方法对比与记忆技巧

查找方法对比

- 位置查找:indexOf() (从左到右)、lastIndexOf() (从右到左)
- 包含检查: includes() (返回布尔值)、startsWith()、endsWith()
 - 正则查找: match() (返回匹配结果)、search() (返回位置)

截取方法对比

- **slice()**: 参数为 (start, end), 支持负数索引
- **substring()**: 参数为 (from, to), 不支持负数索引, 会自动调整参数顺序
- **substr()**: 参数为 (start, length), 第二个参数是长度

转换方法对比

- **大小写转换**: toLowerCase()、toUpperCase()、toLocaleLowerCase()、toLocaleUpperCase()
- **分割与连接**: split() (分割为数组)、concat() (连接字符串)
- **填充方法**: padStart() (开头填充)、padEnd() (末尾填充)

记忆技巧

1. **字符串不可变性**: 所有 String 方法都不会修改原字符串, 而是返回新字符串
2. **参数区别**: slice 和 substring 的第二个参数是结束位置, substr 的第二个参数是长度
3. **静态方法**: fromCharCode() 是静态方法, 直接通过 String 调用
4. **查找返回值**: indexOf 和 lastIndexOf 返回位置, includes、startsWith、endsWith 返回布尔值
5. **替换方法**: replace() 只替换第一个匹配项, 要替换所有匹配项需要使用正则表达式加 g 标志

1.4 Math 对象

1.4.1 Math 对象方法总览

方法	描述	用法示例
abs()	返回数的绝对值	Math.abs(x)
acos()	返回数的反余弦值	Math.acos(x)
asin()	返回数的反正弦值	Math.asin(x)
atan()	返回数的反正切值	Math.atan(x)
atan2()	返回由 x 轴到点 (x,y) 的角度	Math.atan2(y, x)
ceil()	对数进行上舍入	Math.ceil(x)
cos()	返回数的余弦值	Math.cos(x)
exp()	返回 e 的指数	Math.exp(x)
floor()	对数进行下舍入	Math.floor(x)
log()	返回数的自然对数（底为 e）	Math.log(x)
max()	返回 x 和 y 中的最大值	Math.max(x, y, ...)
min()	返回 x 和 y 中的最小值	Math.min(x, y, ...)
pow()	返回 x 的 y 次幂	Math.pow(x, y)
random()	返回 0 1 之间的随机数	Math.random()
round()	把数四舍五入为最接近的整数	Math.round(x)
sin()	返回数的正弦值	Math.sin(x)
sqrt()	返回数的平方根	Math.sqrt(x)
tan()	返回角的正切值	Math.tan(x)
trunc()	返回数字的整数部分	Math.trunc(x)
sign()	返回数字的符号	Math.sign(x)
cbrt()	返回数字的立方根	Math.cbrt(x)
hypot()	返回所有参数的平方和的平方根	Math.hypot(x, y, ...)

clamp()	返回限制在指定范围内的数字	Math.clamp(x, min, max)
fround()	返回数字的最接近的单精度浮点表示	Math.fround(x)
imul()	返回两个数字的类 C 32 位整数乘法结果	Math.imul(x, y)
log10()	返回数字以 10 为底的对数	Math.log10(x)
log2()	返回数字以 2 为底的对数	Math.log2(x)
log1p()	返回数字加 1 后的自然对数	Math.log1p(x)
expm1()	返回 e 的 x 次幂减 1	Math.expm1(x)
cosh()	返回数字的双曲余弦值	Math.cosh(x)
sinh()	返回数字的双曲正弦值	Math.sinh(x)
tanh()	返回数字的双曲正切值	Math.tanh(x)
acosh()	返回数字的反双曲余弦值	Math.acosh(x)
asinh()	返回数字的反双曲正弦值	Math.asinh(x)
atanh()	返回数字的反双曲正切值	Math.atanh(x)

1.4.2 Math 对象方法对比与记忆技巧

数值处理方法对比

- 取整方法: ceil() (向上取整)、floor() (向下取整)、round() (四舍五入)、trunc() (截断小数)

部分)

- 最大值最小值: max()、min()
- 绝对值与符号: abs() (绝对值)、sign() (符号)

数学运算方法对比

- 幂运算: pow() (幂)、sqrt() (平方根)、cbrt() (立方根)
- 指数与对数: exp() (指数)、log() (自然对数)、

log10()、log2()、log1p()、expm1()

- 三角函数: sin()、cos()、tan()、asin()、acos()、atan()、atan2()
- 双曲函数: sinh()、cosh()、tanh()、asinh()、acosh()、atanh()

其他方法对比

- 随机数: random() (0-1 之间)

- 距离计算: hypot() (欧几里得距离)
- 范围限制: clamp()
- 位运算: imul()

记忆技巧

1. **静态方法**: 所有 Math 方法都是静态方法, 直接通过 Math 调用, 不需要创建 Math 对象
2. **参数类型**: 所有 Math 方法都接受数字参数, 非数字参数会被转换为数字
3. **三角函数**: 参数是弧度, 不是角度
4. **随机数生成**: Math.random() 返回 [0,1) 之间的随机数, 要生成指定范围的随机数需要进行计算
5. **取整区别**: ceil 向上, floor 向下, round 四舍五入, trunc 截断

2 Browser 对象参考手册

2.1 Window 对象

2.1.1 Window 对象方法总览

方法	描述	用法示例
alert()	显示带有一段消息和一个确认按钮的警告框	<code>window.alert(message)</code>
blur()	把键盘焦点从顶层窗口移开	<code>window.blur()</code>
clearInterval()	取消由 setInterval() 设置的 timeout	<code>window.clearInterval(id_of_setin</code>
clearTimeout()	取消由 setTimeout() 设置的 timeout	<code>window.clearTimeout(id_of_settim</code>
close()	关闭浏览器窗口	<code>window.close()</code>
confirm()	显示带有一段消息以及确认按钮和取消按钮的对话框	<code>window.confirm(message)</code>
focus()	把键盘焦点给予一个窗口	<code>window.focus()</code>
moveBy()	可相对窗口的当前坐标把它移动指定的像素	<code>window.moveBy(x, y)</code>
moveTo()	把窗口的左上角移动到一个指定的坐标	<code>window.moveTo(x, y)</code>
open()	打开一个新的浏览器窗口或查找一个已命名的窗口	<code>window.open(URL, name, specs, re</code>
print()	打印当前窗口的内容	<code>window.print()</code>
prompt()	显示可提示用户输入的对话框	<code>window.prompt(text, defaultText)</code>
resizeBy()	按照指定的像素调整窗口的大小	<code>window.resizeBy(width, height)</code>

resizeTo()	把窗口的大小调整到指定的宽度和高度	window.resizeTo(width, height)
scrollBy()	按照指定的像素值来滚动内容	window.scrollBy(xnum, ynum)
scrollTo()	把内容滚动到指定的坐标	window.scrollTo(xpos, ypos)
setInterval()	按照指定的周期（以毫秒计）来调用函数或计算表达式	window.setInterval(func, delay, [args])
setTimeout()	在指定的毫秒数后调用函数或计算表达式	window.setTimeout(func, delay, [args])

2.1.2 Window 对象方法对比与记忆技巧

对话框方法对比

- **alert()**: 只有确认按钮，无返回值
- **confirm()**: 有确认和取消按钮，返回布尔值
 - **prompt()**: 有输入框，返回用户输入的字符串

窗口控制方法对比

- **移动窗口**: moveBy()（相对位置）、moveTo()（绝对位置）
 - **调整窗口大小**: resizeBy()（相对大小）、resizeTo()（绝对大小）
- **滚动窗口**: scrollBy()（相对滚动）、scrollTo()（绝对滚动）
 - **焦点控制**: focus()、blur()

定时器方法对比

- **setTimeout()**: 延迟指定时间后执行一次
 - **setInterval()**: 按照指定周期重复执行
- **clearTimeout()**: 清除 setTimeout() 设置的定时器
 - **clearInterval()**: 清除 setInterval() 设置的定时器

记忆技巧

1. **全局对象**: Window 对象是全局对象，所有全局变量和函数都是它的属性和方法
2. **方法简写**: 大多数 Window 方法可以省略 window. 前缀直接调用
3. **定时器返回值**: setTimeout() 和 setInterval() 都返回一个 ID，用于清除定时器
4. **浏览器限制**: 现代浏览器对窗口大小和位置的操作有严格限制，通常只允许操作由脚本打开的窗口

5. 对话框阻塞: `alert()`、`confirm()` 和 `prompt()` 都是阻塞式对话框，会暂停脚本执行

2.2 Navigator 对象

2.2.1 Navigator 对象方法总览

方法	描述	用法示例
javaEnabled()	指定是否在浏览器中启用 Java	<code>navigator.javaEnabled()</code>
taintEnabled()	规定浏览器是否启用数据污点 (data tainting)	<code>navigator.taintEnabled()</code>
sendBeacon()	向服务器异步发送少量数据	<code>navigator.sendBeacon(url, data)</code>
registerProtocolHandler()	注册一个网站作为特定协议的处理程序	<code>navigator.registerProtocolHandler</code>
unregisterProtocolHandler()	取消注册的协议处理程序	<code>navigator.unregisterProtocolHandler</code>
share()	调用设备的原生分享功能	<code>navigator.share(data)</code>
getBattery()	获取电池状态信息	<code>navigator.getBattery()</code>
getUserMedia()	请求用户媒体设备（如摄像头、麦克风）	<code>navigator.getUserMedia(constrain</code>
requestMediaKeySystemAccess()	请求媒体密钥系统访问	<code>navigator.requestMediaKeySystemA</code>
requestMIDIAccess()	请求 MIDI 设备访问	<code>navigator.requestMIDIAccess(opti</code>
requestAnimationFrame()	告诉浏览器您希望执行动画并请求浏览器在下一次重绘之前调用指定的函数来更新动画	<code>requestAnimationFrame(callback)</code>
cancelAnimationFrame()	取消先前通过调用 requestAnimationFrame() 安排的动画帧请求	<code>cancelAnimationFrame(requestID)</code>

2.2.2 Navigator 对象方法对比与记忆技巧

功能检测方法对比

- **javaEnabled()**: 检测 Java 是否启用
- **taintEnabled()**: 检测数据污点是否启用（已废弃）

设备访问方法对比

- **getUserMedia()**: 访问摄像头和麦克风
- **getBattery()**: 获取电池状态
- **requestMIDIAccess()**: 访问 MIDI 设备

通信方法对比

- **sendBeacon()**: 异步发送数据, 适合页面卸载
- **share()**: 调用原生分享功能

动画方法对比

- **requestAnimationFrame()**: 优化动画性能
- **cancelAnimationFrame()**: 取消动画帧请求

记忆技巧

1. **只读属性**: Navigator 对象的大多数属性是只读的, 用于获取浏览器信息
2. **方法用途**: 方法主要用于功能检测、设备访问、通信和动画
3. **异步方法**: `getBattery()`、`getUserMedia()` 等方法返回 Promise 对象
4. **浏览器兼容性**: 不同浏览器对 Navigator 方法的支持程度不同, 使用前应检查兼容性
5. **安全限制**: 访问用户设备的方法通常需要用户授权

2.3 Screen 对象

2.3.1 Screen 对象方法总览

Screen 对象主要提供屏幕相关的属性，方法较少。

方法	描述	用法示例
// 无特定方法	Screen 对象主要通过属性提供屏幕信息	

2.3.2 Screen 对象属性与记忆技巧

Screen 对象主要属性

- **width**: 屏幕宽度（像素）
 - **height**: 屏幕高度（像素）
 - **availWidth**: 可用屏幕宽度（像素）
- **availHeight**: 可用屏幕高度（像素）
 - **colorDepth**: 屏幕颜色深度
 - **pixelDepth**: 屏幕像素深度
 - **orientation**: 屏幕方向

记忆技巧

1. **只读属性**: Screen 对象的所有属性都是只读的
2. **可用空间**: `availWidth` 和 `availHeight` 不包括任务栏等系统界面占用的空间
3. **颜色深度**: `colorDepth` 和 `pixelDepth` 通常是相同的值，表示屏幕支持的颜色位数
4. **屏幕方向**: `orientation` 属性返回一个 `ScreenOrientation` 对象，包含方向信息
5. **用途**: 主要用于响应式设计和适配不同屏幕尺寸

2.4 History 对象

2.4.1 History 对象方法总览

方法	描述	用法示例
back()	加载 history 列表中的前一个 URL	history.back()
forward()	加载 history 列表中的下一个 URL	history.forward()
go()	加载 history 列表中的某个具体页面	history.go(number)
pushState()	向浏览器历史栈添加一个状态	history.pushState(stateObj, title, url)
replaceState()	修改浏览器历史栈中的当前状态	history.replaceState(stateObj, title, url)

2.4.2 History 对象方法对比与记忆技巧

导航方法对比	forward(): 等同于点击浏览器的前进按钮 go(): 可以前进或后退多个页面, 参数为正数前进, 负数后退
back(): 等同于点击浏览器的后退按钮	

状态管理方法对比	发页面刷新
pushState(): 添加新状态到历史栈, 不会触发页面刷新	replaceState(): 替换当前状态, 不会触发页面刷新

记忆技巧

- 历史栈**: History 对象表示浏览器的历史记录栈
- 长度属性**: length 属性表示历史记录的数量
- 状态管理**: pushState() 和 replaceState() 用于实现单页应用的无刷新导航
- popstate 事件**: 当用户点击浏览器的前进或后退按钮时, 会触发 popstate 事件
- 安全限制**: 同源策略限制, 不能访问不同源的历史记录

2.5 Location 对象

2.5.1 Location 对象方法总览

方法	描述	用法示例
<code>assign()</code>	加载新的文档	<code>location.assign(url)</code>
<code>reload()</code>	重新加载当前文档	<code>location.reload(forceGet)</code>
<code>replace()</code>	用新的文档替换当前文档	<code>location.replace(url)</code>
<code>toString()</code>	返回完整的 URL	<code>location.toString()</code>

2.5.2 Location 对象方法对比与记忆技巧

导航方法对比

- **assign()**: 加载新文档, 会在历史记录中添加新条目
- **replace()**: 替换当前文档, 不会在历史记录中添加新条目
- **reload()**: 重新加载当前文档, 可选参数 `forceGet` 为 `true` 时会强制从服务器获取

Location 对象主要属性

- **href**: 完整的 URL
- **protocol**: 协议部分
- **host**: 主机名和端口号
- **hostname**: 主机名
- **port**: 端口号
- **pathname**: 路径部分
- **search**: 查询字符串部分
- **hash**: 哈希部分 (后面的内容)

记忆技巧

1. **属性修改**: 修改 Location 对象的属性会自动导航到新 URL
2. **方法区别**: `assign()` 会留下历史记录, `replace()` 不会
3. **reload() 注意**: `reload()` 可能会丢失页面状态, 使用时需谨慎
4. **URL 解析**: 可以通过 Location 对象的属性轻松解析 URL 的各个部分

5. **同源策略：**Location 对象的某些操作受到同源策略的限制

3 HTML DOM 参考手册

3.1 Document 对象

3.1.1 Document 对象方法总览

方法	描述	用法示例
createElement()	创建元素节点	<code>document.createElement(tagName)</code>
createTextNode()	创建文本节点	<code>document.createTextNode(text)</code>
createAttribute()	创建属性节点	<code>document.createAttribute(name)</code>
createComment()	创建注释节点	<code>document.createComment(text)</code>
createDocumentFragment()	创建文档片段节点	<code>document.createDocumentFragment()</code>
getElementById()	返回对拥有指定 ID 的第一个对象的引用	<code>document.getElementById(id)</code>
getElementsByName()	返回带有指定标签名的对象的集合	<code>document.getElementsByName(tagName)</code>
getElementsByClassName()	返回带有指定类名的对象的集合	<code>document.getElementsByClassName(className)</code>
getElementsByTagName()	返回带有指定名称的对象的集合	<code>document.getElementsByTagName(tagName)</code>
querySelector()	返回文档中匹配指定的 CSS 选择器的第一个元素	<code>document.querySelector(selector)</code>
querySelectorAll()	返回文档中匹配指定的 CSS 选择器的所有元素	<code>document.querySelectorAll(selector)</code>

<code>write()</code>	向文档写 HTML 表达式或 JavaScript 代码	<code>document.write(exp)</code>
<code>writeln()</code>	等同于 <code>write()</code> 方法，不同的是在每个表达式之后写一个换行符	<code>document.writeln(exp)</code>
<code>open()</code>	打开一个流，以收集来自 <code>write()</code> 或 <code>writeln()</code> 方法的输出	<code>document.open()</code>
<code>close()</code>	关闭用 <code>document.open()</code> 方法打开的输出流，并显示选定的数据	<code>document.close()</code>
<code>importNode()</code>	把一个节点从另一个文档复制到当前文档	<code>document.importNode(node, deep)</code>
<code>adoptNode()</code>	从另一个文档中获取一个节点并把它添加到当前文档	<code>document.adoptNode(node)</code>
<code>createEvent()</code>	创建一个事件对象	<code>document.createEvent(type)</code>
<code>dispatchEvent()</code>	向文档触发一个事件	<code>document.dispatchEvent(event)</code>
<code>execCommand()</code>	执行一个对当前文档，当前选择或给出范围的命令	<code>document.execCommand(command, sh</code>
<code>getSelection()</code>	返回一个 <code>Selection</code> 对象，表示用户选择的文本范围或光标的当前位置	<code>document.getSelection()</code>
<code>hasFocus()</code>	返回布尔值，指示文档中是否有元素获得焦点	<code>document.hasFocus()</code>
<code>normalize()</code>	删除空文本节点，并合并相邻的文本节点	<code>element.normalize()</code>
<code>normalizeDocument()</code>	对文档进行 <code>normalize</code> 操作	<code>document.normalizeDocument()</code>

3.1.2 Document 对象方法对比与记忆技巧

元素获取方法对比

- 按 ID 获取: `getElementById()` (返回单个元素)
- 按标签名获取: `getElementsByTagName()` (返回集合)
- 按类名获取: `getElementsByClassName()` (返回集合)

- 按名称获取: `getElementsByName()` (返回集合)
- 按选择器获取: `querySelector()` (返回第一个匹配元素)、`querySelectorAll()` (返回所有匹配元素)

节点创建方法对比

- 元素节点: `createElement()`
- 文本节点: `createTextNode()`
- 属性节点: `createAttribute()`
- 注释节点: `createComment()`
- 文档片段: `createDocumentFragment()`

文档写入方法对比

- `write()`: 写入内容
- `writeln()`: 写入内容并添加换行符
- `open()`: 打开输出流
- `close()`: 关闭输出流

记忆技巧

1. 返回值区别: `getElementById()` 返回单个元素, 其他 `getElements*` 方法返回集合
2. 选择器优势: `querySelector()` 和 `querySelectorAll()` 支持 CSS 选择器, 更灵活
3. 文档片段: `createDocumentFragment()` 用于批量操作 DOM, 提高性能
4. 事件创建: `createEvent()` 用于创建自定义事件
5. 安全注意: `write()` 方法可能导致 XSS 攻击, 使用时需谨慎

3.2 Element 对象

3.2.1 Element 对象方法总览

方法	描述	用法示例
getAttribute()	返回元素的指定属性值	<code>element.getAttribute(attribute)</code>
setAttribute()	设置元素的指定属性值	<code>element.setAttribute(attribute, value)</code>
removeAttribute()	从元素中移除指定的属性	<code>element.removeAttribute(attribute)</code>
hasAttribute()	检查元素是否具有指定的属性	<code>element.hasAttribute(attribute)</code>
getAttributeNode()	返回指定的属性节点	<code>element.getAttributeNode(attributeName)</code>
setAttributeNode()	设置或更改指定的属性节点	<code>element.setAttributeNode(attributeNode)</code>
removeAttributeNode()	从元素中移除指定的属性节点	<code>element.removeAttributeNode(attributeNode)</code>
getElementsByTagName()	返回带有指定标签名的所有子元素的集合	<code>element.getElementsByTagName(tagName)</code>
getElementsByClassName()	返回带有指定类名的所有子元素的集合	<code>element.getElementsByClassName(className)</code>
querySelector()	返回元素中匹配指定的 CSS 选择器的第一个元素	<code>element.querySelector(selector)</code>
querySelectorAll()	返回元素中匹配指定的 CSS 选择器的所有元素	<code>element.querySelectorAll(selector)</code>
appendChild()	向元素添加新的子节点，作为最后一个子节点	<code>element.appendChild(node)</code>
insertBefore()	在指定的子节点前插入新的子节点	<code>element.insertBefore(newNode, existingNode)</code>
removeChild()	从元素中移除子节点	<code>element.removeChild(node)</code>

<code>replaceChild()</code>	替换元素中的子节点	<code>element.replaceChild(newNode, oldNode)</code>
<code>hasChildNodes()</code>	检查元素是否拥有子节点	<code>element.hasChildNodes()</code>
<code>cloneNode()</code>	复制元素节点	<code>element.cloneNode(deep)</code>
<code>normalize()</code>	删除空文本节点，并合并相邻的文本节点	<code>element.normalize()</code>
<code>scrollIntoView()</code>	滚动元素的父容器，使元素对用户可见	<code>element.scrollIntoView(alignToTop)</code>
<code>focus()</code>	使元素获得焦点	<code>element.focus()</code>
<code>blur()</code>	使元素失去焦点	<code>element.blur()</code>
<code>click()</code>	模拟对元素的点击	<code>element.click()</code>
<code>setAttributeNS()</code>	设置带有命名空间的属性	<code>element.setAttributeNS(namespaceURI, localName, value)</code>
<code>getAttributeNS()</code>	获取带有命名空间的属性	<code>element.getAttributeNS(namespaceURI, localName)</code>
<code>removeAttributeNS()</code>	移除带有命名空间的属性	<code>element.removeAttributeNS(namespaceURI, localName)</code>
<code>hasAttributeNS()</code>	检查元素是否具有带有命名空间的属性	<code>element.hasAttributeNS(namespaceURI, localName)</code>

3.2.2 Element 对象方法对比与记忆技巧

属性操作方法对比

- `getAttribute()`: 获取属性值
- `setAttribute()`: 设置属性值
- `removeAttribute()`: 移除属性
- `hasAttribute()`: 检查属性是否存在
- `getAttributeNode()`: 获取属性节点
- `setAttributeNode()`: 设置属性节点
- `removeAttributeNode()`: 移除属性节点

子节点操作方法对比

- `appendChild()`: 添加到末尾
- `insertBefore()`: 插入到指定节点前

- **removeChild()**: 移除子节点
- **replaceChild()**: 替换子节点
- **hasChildNodes()**: 检查是否有子节点

元素查找方法对比

子元素

- **getElementsByTagName()**: 按标签名查找子元素
- **getElementsByClassName()**: 按类名查找子元素
- **querySelector()**: 按选择器查找第一个匹配子元素
- **querySelectorAll()**: 按选择器查找所有匹配子元素

记忆技巧

1. **节点操作**: `appendChild()`、`insertBefore()`、`removeChild()` 和 `replaceChild()` 是基本的 DOM 操作方法
2. **克隆节点**: `cloneNode()` 的 `deep` 参数为 `true` 时会克隆所有子节点
3. **滚动方法**: `scrollIntoView()` 用于将元素滚动到可见区域
4. **焦点操作**: `focus()` 和 `blur()` 用于管理元素焦点
5. **命名空间**: 带有 `NS` 后缀的方法用于处理 XML 命名空间

3.3 Event 对象

3.3.1 Event 对象方法总览

方法	描述	用法示例
preventDefault()	阻止事件的默认行为	<code>event.preventDefault()</code>
stopPropagation()	阻止事件冒泡到父元素	<code>event.stopPropagation()</code>
stopImmediatePropagation()	阻止事件冒泡到父元素，同时阻止元素上的其他事件监听器被触发	<code>event.stopImmediatePropagation()</code>
initEvent()	初始化事件对象的属性	<code>event.initEvent(type, bubbles, cancelable)</code>
initMouseEvent()	初始化鼠标事件对象的属性	<code>event.initMouseEvent(type, bubbles, cancelable, view, detail, screenX, screenY, clientX, clientY, button, relatedTarget)</code>
initKeyboardEvent()	初始化键盘事件对象的属性	<code>event.initKeyboardEvent(type, bubbles, cancelable, view, detail, keyCode, location)</code>
initUIEvent()	初始化 UI 事件对象的属性	<code>event.initUIEvent(type, bubbles, cancelable, view, detail, propertyName, nodeType, relatedTarget)</code>
composedPath()	返回事件的路径，即事件经过的 DOM 节点列表	<code>event.composedPath()</code>
getModifierState()	返回一个布尔值，表示指定的修饰键是否被按下	<code>event.getModifierState(key)</code>

3.3.2 Event 对象方法对比与记忆技巧

事件控制方法对比	• preventDefault(): 阻止默认行为，如链接跳转、表单提交等 • stopPropagation(): 阻止事件冒泡，但不阻止当前元素的其他监听器 • stopImmediatePropagation(): 阻止事件冒泡，同时阻止当前元素的其他监听器
事件初始化方法对比	• initEvent(): 初始化通用事件 • initMouseEvent(): 初始化鼠标事件 • initKeyboardEvent(): 初始化键盘事件 • initUIEvent(): 初始化 UI 事件

记忆技巧

1. **事件对象**: Event 对象在事件监听器被触发时自动创建并传递给监听器函数
2. **事件冒泡**: 事件默认会从目标元素冒泡到文档根元素
3. **默认行为**: 很多 HTML 元素都有默认行为, 如点击链接会跳转
4. **事件委托**: 利用事件冒泡可以实现事件委托, 提高性能
5. **现代事件 API**: 现代浏览器推荐使用 `addEventListener()` 方法添加事件监听器

4 HTML DOM 元素对象参考手册

4.1 Form 对象

4.1.1 Form 对象方法总览

方法	描述	用法示例
submit()	提交表单	form.submit()
reset()	重置表单	form.reset()
checkValidity()	检查表单是否有效	form.checkValidity()
reportValidity()	检查表单是否有效，并显示错误信息	form.reportValidity()
requestSubmit()	请求提交表单	form.requestSubmit(submitter)

4.1.2 Form 对象方法对比与记忆技巧

表单操作方法对比

- 提交操作: submit()（直接提交）、requestSubmit()（模拟提交按钮点击）
- 重置操作: reset()
 - 验证操作: checkValidity()（返回布尔值）、reportValidity()（显示错误信息）

记忆技巧

- 提交区别: submit() 不会触发 submit 事件，requestSubmit() 会触发
- 验证方法: checkValidity() 用于静默验证，reportValidity() 用于显示验证错误
- 表单元素: Form 对象包含表单中的所有输入元素
- 表单属性: 可以通过 Form 对象的属性访问表单的各种信息
- 事件处理: 表单可以监听 submit、reset 等事件

4.2 Input 对象

4.2.1 Input 对象方法总览

方法	描述	用法示例
focus()	使输入框获得焦点	<code>input.focus()</code>
blur()	使输入框失去焦点	<code>input.blur()</code>
select()	选择输入框中的文本	<code>input.select()</code>
setSelection-Range()	设置输入框中选择的文本范围	<code>input.setSelectionRange(start, end)</code>
checkValidity()	检查输入值是否有效	<code>input.checkValidity()</code>
reportValidity()	检查输入值是否有效，并显示错误信息	<code>input.reportValidity()</code>
stepUp()	增加输入值	<code>input.stepUp(step)</code>
stepDown()	减少输入值	<code>input.stepDown(step)</code>

4.2.2 Input 对象方法对比与记忆技巧

焦点操作方法对比	• <code>focus()</code>: 使元素获得焦点 • <code>blur()</code>: 使元素失去焦点
文本选择方法对比	• <code>select()</code>: 选择所有文本 • <code>setSelectionRange()</code>: 选择指定范围的文本
值操作方法对比	• <code>stepUp()</code>: 增加值 • <code>stepDown()</code>: 减少值
记忆技巧	1. 输入类型: 不同类型的输入元素支持不同的方法 2. 验证方法: <code>checkValidity()</code> 和 <code>reportValidity()</code> 用于表单验证

3. 文本操作: `select()` 和 `setSelectionRange()` 用于文本选择
4. 值调整: `stepUp()` 和 `stepDown()` 主要用于数字、日期等类型的输入
5. 事件处理: 输入元素可以监听 `focus`、`blur`、`input`、`change` 等事件

4.3 Select 对象

4.3.1 Select 对象方法总览

方法	描述	用法示例
add()	向下拉列表添加一个选项	<code>select.add(option, before)</code>
remove()	从下拉列表中删除一个选项	<code>select.remove(index)</code>
focus()	使下拉列表获得焦点	<code>select.focus()</code>
blur()	使下拉列表失去焦点	<code>select.blur()</code>
checkValidity()	检查选择是否有效	<code>select.checkValidity()</code>

4.3.2 Select 对象方法对比与记忆技巧

- 选项操作方法对比
- 添加选项: `add()`
 - 删除选项: `remove()`

- 记忆技巧
1. 选项管理: `Select` 对象用于管理下拉列表的选项
 2. 选项对象: `add()` 方法需要传入 `Option` 对象
 3. 索引参数: `remove()` 方法的参数是选项的索引
 4. 选中状态: 可以通过 `value` 属性或 `selectedIndex` 属性获取选中的值
 5. 事件处理: `Select` 对象可以监听 `change` 事件

4.4 Image 对象

4.4.1 Image 对象方法总览

方法	描述	用法示例
focus()	使图像获得焦点	<code>image.focus()</code>
blur()	使图像失去焦点	<code>image.blur()</code>

4.4.2 Image 对象方法对比与记忆技巧

记忆技巧

1. **图像操作：** Image 对象主要用于操作图像元素
2. **属性为主：** Image 对象的属性比方法更多
3. **加载事件：** 可以监听 load、error 等事件
4. **动态创建：** 可以通过 JavaScript 动态创建 Image 对象
5. **预加载：** 常用于图像预加载

4.5 Table 对象

4.5.1 Table 对象方法总览

方法	描述	用法示例
createTHead()	创建表格的表头部分	<code>table.createTHead()</code>
deleteTHead()	删除表格的表头部分	<code>table.deleteTHead()</code>
createTBody()	创建表格的主体部分	<code>table.createTBody()</code>
createTFoot()	创建表格的页脚部分	<code>table.createTFoot()</code>
deleteTFoot()	删除表格的页脚部分	<code>table.deleteTFoot()</code>
insertRow()	在表格中插入一行	<code>table.insertRow(index)</code>
deleteRow()	从表格中删除一行	<code>table.deleteRow(index)</code>

4.5.2 Table 对象方法对比与记忆技巧

表格结构操作方法对比

- 表头操作: `createTHead()`、`deleteTHead()`
- 表体操作: `createTBody()`
 - 表脚操作: `createTFoot()`、`deleteTFoot()`
 - 行操作: `insertRow()`、`deleteRow()`

记忆技巧

1. 结构操作: Table 对象的方法主要用于操作表格的结构
2. 返回值: `create*` 方法返回创建的元素
3. 索引参数: `insertRow()` 和 `deleteRow()` 的 `index` 参数是可选的
4. 行操作: 插入和删除行后, 表格的行数会自动调整
5. 单元格操作: 通过返回的行对象可以进一步操作单元格

5 总结

本参考手册基于 runoob.com 教程，详细介绍了 JavaScript 和 HTML DOM 中各种对象的方法、用法和对比记忆技巧。通过本手册，您可以：

1. 快速查阅各种对象的方法及其用法
2. 对比相似方法的区别，避免混淆
3. 掌握记忆技巧，提高学习效率
4. 了解对象之间的关系，形成完整的知识体系

希望本参考手册对您的 JavaScript 和 HTML DOM 学习有所帮助！